DDoS Attack Detection Using Deep Learning

LSTM + CIC-DDoS2019 + FastAPI: explanation and complete build guide, from scratch

Stack	Python 3.10/3.11, TensorFlow/Keras, scikit-learn, pandas, NumPy, FastAPI, Uvicorn
Dataset	CIC-DDoS2019 (Canadian Institute for Cybersecurity, Univ. of New Brunswick)
Goal	Classify network flows as BENIGN or DDoS attack and serve predictions through a REST API
Output	Trained model, scaler, evaluation report, running API at /predict

Part 1. What this project is

1.1 The problem

A Distributed Denial of Service (DDoS) attack floods a server or network with traffic from many sources so that legitimate users cannot be served. Examples in CIC-DDoS2019 include SYN flood, UDP flood, UDP-Lag, DNS, LDAP, MSSQL, NetBIOS, NTP, SNMP, SSDP and TFTP reflection/amplification attacks. Rule-based defences (rate limits, static signatures) struggle with new or mixed attack patterns. A learned model can pick up statistical patterns in traffic and flag attacks automatically.

1.2 The idea in one paragraph

Network traffic is captured and summarised into **flows** (one row per connection: packet counts, byte counts, inter-arrival times, TCP flag counts, and so on, roughly 80 numeric features produced by CICFlowMeter). You train an **LSTM** (Long Short-Term Memory network, a recurrent neural network good at sequence patterns) on labelled flows so it learns to separate benign from attack traffic. The trained model is then wrapped in a **FastAPI** service: a client sends flow features as JSON and gets back a label and probability in milliseconds.

1.3 Pipeline overview

Stage	What happens	Why it matters
1. Data	Download and sample CIC-DDoS2019 CSVs	Raw data is tens of GB; you need a manageable subset
2. Preprocessing	Strip column names, drop identifiers, remove inf/NaN/duplicates, label BENIGN=0 / ATTACK=1	Dirty data breaks training and leaks IP-specific shortcuts
3. Split	Stratified train / validation / test	Honest evaluation on unseen data
4. Feature scaling	StandardScaler fitted on train only	LSTMs train badly on features with wildly different ranges
5. Class balancing	Class weights (or resampling on train only)	Attack and benign counts are very unequal; otherwise the model ignores the minority class
6. Model	Stacked LSTM + Dropout + sigmoid output	Learns patterns across the feature sequence

Stage	What happens	Why it matters
7. Evaluation	Precision, recall, F1, ROC-AUC, confusion matrix	Accuracy alone is misleading on imbalanced data
8. Deployment	Save model + scaler, serve via FastAPI	Turns a notebook into a usable real-time service

1.4 An honest note on the LSTM choice

CIC-DDoS2019 rows are *flow-level* tabular features, not raw packet sequences. If you feed each flow to an LSTM as a single time step, the LSTM behaves much like a dense network. That is acceptable and is what most published student and paper implementations do, but be ready to explain it in interviews. Section 9 shows how to make the temporal part real by building **windows of consecutive flows**. This guide implements the simple version first, then the windowed version as an upgrade.

Part 2. Environment and project layout

Step 1. Install tools

- Python 3.10 or 3.11 (TensorFlow 2.15+ supports these; check the TensorFlow install page if you use 3.12+).
- A machine with at least 16 GB RAM. A GPU helps but is not required if you sample the data.
- VS Code or PyCharm, and Git.

Step 2. Create the project and virtual environment

```
mkdir ddos-lstm && cd ddos-lstm
python -m venv .venv
# Windows:   .venv\Scripts\activate
# Linux/Mac: source .venv/bin/activate

pip install --upgrade pip
pip install tensorflow pandas numpy scikit-learn matplotlib seaborn joblib \
            fastapi "uvicorn[standard]" pydantic requests
pip freeze > requirements.txt
```

Step 3. Create the folder structure

```
ddos-lstm/
|-- data/
|   |-- raw/            # downloaded CIC-DDoS2019 CSV files go here
|   `-- processed/      # dataset.npz, sample_flows.json (created by scripts)
|-- models/             # lstm_ddos.keras, scaler.joblib, features.json
|-- reports/            # plots and metrics
|-- src/
|   |-- preprocess.py
|   |-- train.py
|   `-- evaluate.py
|-- app/
|   `-- main.py         # FastAPI service
|-- tests/
|   `-- client.py
`-- requirements.txt
```

Create the empty folders with: `mkdir -p data/raw data/processed models reports src app tests` (on Windows PowerShell use `mkdir` for each, or create them in your editor).

Part 3. Get the dataset

Step 4. Download CIC-DDoS2019

- Go to the Canadian Institute for Cybersecurity dataset page for DDoS 2019 (search: *CIC-DDoS2019 UNB*). The site asks for a short registration form before download.
- You want the **CSV files of labelled flows** (the CICFlowMeter output), not the raw PCAPs. They come in two training/testing day groups (01-12 and 03-11).
- Unzip the CSVs into `data/raw/`. Mirrors also exist on Kaggle; check the file layout and license.
- The full data is many millions of rows and tens of GB. Do **not** load it all; the preprocessing script samples a fraction.

Note: Column names in these files often have leading spaces (for example ' Label', ' Source IP'). The script strips them. Label values include BENIGN and attack names such as DrDoS_DNS, Syn, UDP, UDPLag, DrDoS_NTP, and so on. Always print `df['Label'].value_counts()` to see exactly what your files contain.

Part 4. Preprocessing, scaling and balancing

Step 5. Write src/preprocess.py

This script loads a sample of every CSV, cleans it, creates a binary label, splits the data, fits the scaler on the training split only, and saves everything the later steps need.

```
# src/preprocess.py
import glob, json, os
import numpy as np
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

RAW_DIR = "data/raw"
OUT_DIR = "data/processed"
SAMPLE_FRAC = 0.05          # raise if you have more RAM, lower if you run out of memory
SEED = 42
CLIP = 10.0                 # clip scaled values to [-CLIP, CLIP] to tame extreme outliers

# identifier / leakage-prone columns: the model must learn traffic behaviour, not specific IPs
DROP_COLS = ["Unnamed: 0", "Flow ID", "Source IP", "Source Port", "Destination IP",
             "Destination Port", "Timestamp", "SimillarHTTP"]

def load_data():
    frames = []
    files = sorted(glob.glob(f"{RAW_DIR}/**/*.csv", recursive=True))
    assert files, "No CSV files found in data/raw"
    for f in files:
        print("Reading", f)
        for chunk in pd.read_csv(f, chunksize=200_000, low_memory=False):
            chunk.columns = chunk.columns.str.strip()
            frames.append(chunk.sample(frac=SAMPLE_FRAC, random_state=SEED))
    return pd.concat(frames, ignore_index=True)

def clean(df):
    df = df.drop(columns=[c for c in DROP_COLS if c in df.columns])
    df = df.replace([np.inf, -np.inf], np.nan).dropna().drop_duplicates()
    return df

def main():
    os.makedirs(OUT_DIR, exist_ok=True)
    os.makedirs("models", exist_ok=True)

    df = clean(load_data())
    print("Label counts:\n", df["Label"].value_counts())

    # binary target: 0 = BENIGN, 1 = any DDoS attack
    y = (df["Label"].astype(str).str.strip().str.upper() != "BENIGN").astype(int).values
    X = df.drop(columns=["Label"]).select_dtypes(include=[np.number])
    X = X.loc[:, X.nunique() > 1]          # drop constant columns
    features = list(X.columns)
```

```
    print("Features kept:", len(features))
    print("Attack ratio:", y.mean().round(4))

    # 70 / 15 / 15 stratified split
    X_train, X_tmp, y_train, y_tmp = train_test_split(
        X, y, test_size=0.30, stratify=y, random_state=SEED)
    X_val, X_test, y_val, y_test = train_test_split(
        X_tmp, y_tmp, test_size=0.50, stratify=y_tmp, random_state=SEED)

    # keep a few raw (unscaled) test rows to try the API later
    X_test.head(5).to_json(f"{OUT_DIR}/sample_flows.json", orient="records")

    # fit scaler on TRAIN ONLY (fitting on all data leaks test statistics into training)
    scaler = StandardScaler().fit(X_train.astype("float64"))
    def prep(a):
        z = scaler.transform(a.astype("float64"))
        return np.clip(z, -CLIP, CLIP).astype("float32")

    np.savez_compressed(
        f"{OUT_DIR}/dataset.npz",
        X_train=prep(X_train), y_train=y_train,
        X_val=prep(X_val),     y_val=y_val,
        X_test=prep(X_test),   y_test=y_test)

    joblib.dump(scaler, "models/scaler.joblib")
    json.dump(features, open("models/features.json", "w"))
    print("Saved dataset + scaler + feature list")

if __name__ == "__main__":
    main()
```

Run it:

```
python src/preprocess.py
```

What each preprocessing choice does

- **Strip column names**: avoids KeyErrors from ' Label'.
- **Drop IPs, ports, timestamp, Flow ID**: otherwise the model memorises who sent the traffic instead of how it behaves, and your accuracy looks great but fails on new networks.
- **Remove inf/NaN/duplicates**: columns like Flow Bytes/s contain infinities; duplicates inflate scores when copies land in both train and test.
- **Binary label**: BENIGN vs ATTACK is the clearest first target. A multi-class option is in Section 10.
- **Scaler on train only**: a very common mistake is fitting on the whole dataset, which leaks information.
- **Clipping**: CIC flow features contain huge outliers; clipping scaled values keeps LSTM training stable.

Step 6. Class balancing

Check the ratio printed above. CIC-DDoS2019 is dominated by attack flows, with very few BENIGN flows. Three options, in order of simplicity:

Method	How	Caution
Class weights (used here)	Pass class_weight to model.fit so errors on the rare class cost more	Simple, no data duplication
Undersampling	Randomly drop majority-class rows in the training set	Throws away data; do it on train only
SMOTE / oversampling	pip install imbalanced-learn; SMOTE().fit_resample(X_train, y_train)	Apply to the training split only, never val/test, and before windowing

Note: Whatever balancing you use, keep the validation and test sets at their **natural** distribution so the metrics reflect reality.

Part 5. Build, train and evaluate the LSTM

Step 7. Write src/train.py

Keras LSTMs expect input shape (samples, timesteps, features). With flow-level data we use one time step (shape n, 1, F). Section 9 replaces this with real windows.

```
# src/train.py
import numpy as np
import tensorflow as tf
from sklearn.utils.class_weight import compute_class_weight
from tensorflow.keras import layers, models, callbacks, metrics, optimizers

tf.keras.utils.set_random_seed(42)

d = np.load("data/processed/dataset.npz")
X_train, y_train = d["X_train"], d["y_train"]
X_val,   y_val   = d["X_val"],   d["y_val"]

# reshape to (samples, timesteps=1, features)
X_train = X_train[:, None, :]
X_val   = X_val[:, None, :]
n_features = X_train.shape[2]

# class weights from the training labels
cw = compute_class_weight("balanced", classes=np.array([0, 1]), y=y_train)
class_weight = {0: cw[0], 1: cw[1]}
print("class_weight:", class_weight)

model = models.Sequential([
    layers.Input(shape=(1, n_features)),
    layers.LSTM(64, return_sequences=True),
    layers.Dropout(0.3),
    layers.LSTM(32),
    layers.Dropout(0.3),
    layers.Dense(32, activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])

model.compile(
    optimizer=optimizers.Adam(1e-3),
    loss="binary_crossentropy",
    metrics=["accuracy",
             metrics.Precision(name="precision"),
             metrics.Recall(name="recall"),
             metrics.AUC(name="auc")],
)
model.summary()

cbs = [
    callbacks.EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True),
    callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2, min_lr=1e-5),
    callbacks.ModelCheckpoint("models/lstm_ddos.keras", monitor="val_loss", save_best_only=True),
]

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=30, batch_size=512,
    class_weight=class_weight,
    callbacks=cbs, verbose=2,
)

model.save("models/lstm_ddos.keras")
np.save("reports/history.npy", history.history)
print("Training finished, model saved")

python src/train.py
```

Step 8. Write src/evaluate.py

```
# src/evaluate.py
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, roc_curve, precision_recall_fscore_support)

d = np.load("data/processed/dataset.npz")
X_test, y_test = d["X_test"][:, None, :], d["y_test"]

model = tf.keras.models.load_model("models/lstm_ddos.keras")
prob = model.predict(X_test, batch_size=1024).ravel()
pred = (prob >= 0.5).astype(int)

print(classification_report(y_test, pred, target_names=["BENIGN", "ATTACK"], digits=4))
print("ROC-AUC:", round(roc_auc_score(y_test, prob), 4))

# confusion matrix
cm = confusion_matrix(y_test, pred)
plt.figure(figsize=(4.5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["BENIGN", "ATTACK"], yticklabels=["BENIGN", "ATTACK"])
plt.xlabel("Predicted"); plt.ylabel("Actual"); plt.title("Confusion matrix")
plt.tight_layout(); plt.savefig("reports/confusion_matrix.png", dpi=150); plt.close()

# ROC curve
fpr, tpr, _ = roc_curve(y_test, prob)
plt.figure(figsize=(5, 4)); plt.plot(fpr, tpr); plt.plot([0, 1], [0, 1], "--")
plt.xlabel("False positive rate"); plt.ylabel("True positive rate"); plt.title("ROC curve")
plt.tight_layout(); plt.savefig("reports/roc_curve.png", dpi=150); plt.close()

# training curves
h = np.load("reports/history.npy", allow_pickle=True).item()
plt.figure(figsize=(5, 4))
plt.plot(h["loss"], label="train"); plt.plot(h["val_loss"], label="val")
plt.legend(); plt.title("Loss"); plt.tight_layout()
plt.savefig("reports/loss_curve.png", dpi=150); plt.close()

python src/evaluate.py
```

How to read the results

- **Recall on ATTACK**: share of real attacks caught. Missing attacks is usually the costly error.
- **Precision on ATTACK**: share of alerts that are real. Low precision means false alarms.
- **BENIGN recall**: because benign is the rare class here, check this one carefully; it can look bad even when overall accuracy is 99%.
- **F1 and ROC-AUC**: balanced summaries. Report them instead of accuracy alone.
- **Suspiciously perfect scores (99.99%)**: CIC-DDoS2019 is known to be very separable, and duplicated rows or identifier columns can inflate it further. State this honestly and test on the other day's file (train on 01-12, test on 03-11) for a tougher, more credible result.

Part 6. Deploy with FastAPI

Step 9. Write app/main.py

The API loads the model, scaler and feature list once at startup, validates that every required feature is present, applies the same scaling and clipping as training, and returns a label with a probability.

```
# app/main.py
import json
import time
```

```
import numpy as np
import joblib
import tensorflow as tf
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

CLIP = 10.0
THRESHOLD = 0.5

app = FastAPI(title="DDoS Detection API", version="1.0")

model = tf.keras.models.load_model("models/lstm_ddos.keras")
scaler = joblib.load("models/scaler.joblib")
FEATURES = json.load(open("models/features.json"))

class Flow(BaseModel):
    features: dict[str, float]

class Batch(BaseModel):
    flows: list[dict[str, float]]

def to_input(rows):
    for i, r in enumerate(rows):
        missing = [f for f in FEATURES if f not in r]
        if missing:
            raise HTTPException(422, f"Row {i} missing {len(missing)} features, e.g. {missing[:5]}")
    X = np.array([[r[f] for f in FEATURES] for r in rows], dtype="float64")
    X = np.clip(scaler.transform(X), -CLIP, CLIP).astype("float32")
    return X[:, None, :]                    # (n, 1, F)  same shape as training

def label_of(p):
    return "ATTACK" if p >= THRESHOLD else "BENIGN"

@app.get("/health")
def health():
    return {"status": "ok", "n_features": len(FEATURES)}

@app.get("/features")
def features():
    return {"features": FEATURES}

@app.post("/predict")
def predict(flow: Flow):
    t0 = time.perf_counter()
    p = float(model.predict(to_input([flow.features]), verbose=0)[0][0])
    return {"label": label_of(p), "attack_probability": round(p, 4),
            "latency_ms": round((time.perf_counter() - t0) * 1000, 2)}

@app.post("/predict_batch")
def predict_batch(batch: Batch):
    probs = model.predict(to_input(batch.flows), verbose=0).ravel()
    return {"results": [{"label": label_of(float(p)), "attack_probability": round(float(p), 4)}
                        for p in probs]}
```

Step 10. Run and test the API

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Open `http://127.0.0.1:8000/docs` for the automatic Swagger UI. Then write `tests/client.py` to send the saved sample rows:

```
# tests/client.py
import json, requests

rows = json.load(open("data/processed/sample_flows.json"))
r = requests.post("http://127.0.0.1:8000/predict", json={"features": rows[0]})
print(r.status_code, r.json())

r = requests.post("http://127.0.0.1:8000/predict_batch", json={"flows": rows})
print(r.status_code, r.json())

python tests/client.py
```

Note: Pydantic requires JSON numbers. If a sample row contains NaN, it would have been removed during cleaning, so the saved rows are safe. Remember the first `/predict` call is slower because TensorFlow warms up.

Part 7. Make it behave like "real-time"

The API classifies *flow records*. To feed it live traffic you need a flow generator in front of it:

- Run **CICFlowMeter** (the same tool used to build the dataset) on a network interface or PCAP replay to produce flow CSV rows.
- Write a small script that tails the CSV (or reads flows from the tool's output), strips column names the same way, keeps only the features in features.json, and POSTs batches to /predict_batch.
- Print or log alerts when attack_probability is above your threshold. Test only on traffic you own or a lab network, using replayed PCAPs of the dataset.

```
# tests/stream_demo.py - simulate a live stream from the held-out test CSV
import time, json, requests
rows = json.load(open("data/processed/sample_flows.json"))
for r in rows:
    res = requests.post("http://127.0.0.1:8000/predict", json={"features": r}).json()
    print(res)
    time.sleep(0.5)
```

Part 8. Optional: Docker

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /code
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
COPY models ./models
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

docker build -t ddos-api .
docker run -p 8000:8000 ddos-api
```

Part 9. Upgrade: real sequences for the LSTM

To give the LSTM genuine temporal context, group consecutive flows into windows and label each window by its last flow (or by majority). This requires keeping flow order, so change preprocessing: keep the Timestamp column long enough to sort, do **not** randomly sample rows (take contiguous chunks instead), and split train/val/test **by time or by file**, not randomly, to avoid overlapping windows leaking between splits.

```
import numpy as np

def make_windows(X, y, window=10, stride=1):
    """X: (n, F) scaled features in time order. Returns (m, window, F) and (m,)"""
    idx = np.arange(0, len(X) - window + 1, stride)
    Xw = np.stack([X[i:i + window] for i in idx])
    yw = np.array([y[i + window - 1] for i in idx])
    return Xw, yw

# then in train.py: remove the [:, None, :] reshape and use Input(shape=(window, n_features))
# in the API: accept a list of `window` flows per request and reshape to (1, window, F)
```

Part 10. Upgrade ideas

- **Multi-class detection**: use LabelEncoder on the attack names, set the final layer to Dense(num_classes, activation='softmax') and the loss to sparse_categorical_crossentropy; report per-class F1.

- **Feature selection**: rank features with a RandomForest or mutual information and keep the top 30 to 40; compare accuracy and latency.
- **Baselines**: train RandomForest/XGBoost and a plain MLP; show the LSTM against them in a results table. Reviewers value this.
- **Threshold tuning**: choose the decision threshold from the precision-recall curve to match how costly misses versus false alarms are.
- **Cross-day testing**: train on one day's data, test on the other to measure generalisation.
- **Monitoring**: log prediction counts and latency, and add an API key header for access control.

Part 11. Common problems and fixes

Problem	Likely cause	Fix
MemoryError while loading	Full CSVs read at once	Lower SAMPLE_FRAC; keep chunked reading
KeyError: 'Label'	Leading space in column names	df.columns = df.columns.str.strip() (already in script)
Loss is NaN	inf/NaN or huge values left in data	Re-run cleaning; keep clipping; lower learning rate
99.99% accuracy, every class	Duplicates, identifier columns or very separable data	Drop IPs/ports/timestamp, dedupe, test cross-day
Model predicts only one class	Extreme imbalance	Check class_weight, look at recall/F1, try resampling on train
API 422 error	Missing or misnamed features	Call GET /features and send exactly those keys
API results differ from notebook	Scaling or clipping mismatch	Use the same scaler.joblib and CLIP value in both places
TensorFlow import errors	Python version mismatch	Use Python 3.10/3.11 in a fresh venv

Part 12. Writing it up (README and interview points)

- **README sections**: problem, dataset, pipeline diagram, how to run (preprocess, train, evaluate, serve), results table, limitations.
- **Be ready to explain**: why drop IPs; why scale on train only; why accuracy is misleading; why class weights; what an LSTM adds over a dense network on flow features; what real-time means in this system (flow-level classification after flows complete, not per-packet).
- **Limitations to state honestly**: lab-generated dataset, flows must be built by a separate tool, attack types outside the training set may be missed, and the model needs retraining as traffic changes.
- **Resume line you can now defend**: report your actual numbers, for example "F1 of X on cross-day test set", instead of a generic claim.

Build checklist

#	Task	Done
1	Environment + folders created	[ ]
2	CIC-DDoS2019 CSVs placed in data/raw	[ ]
3	preprocess.py runs, dataset.npz + scaler + features saved	[ ]

#	Task	Done
4	train.py runs, models/lstm_ddos.keras saved	[ ]
5	evaluate.py produces report + plots	[ ]
6	FastAPI /health, /predict, /predict_batch working	[ ]
7	client.py returns sensible labels	[ ]
8	Baselines + cross-day test + README written	[ ]